

AI ASSISTED CODING ASSIGNMENT-1

Task 1: AI-Generated Logic Without Modularization (String Reversal Without Functions)

❖ Scenario

You are developing a basic text-processing utility for a messaging application.

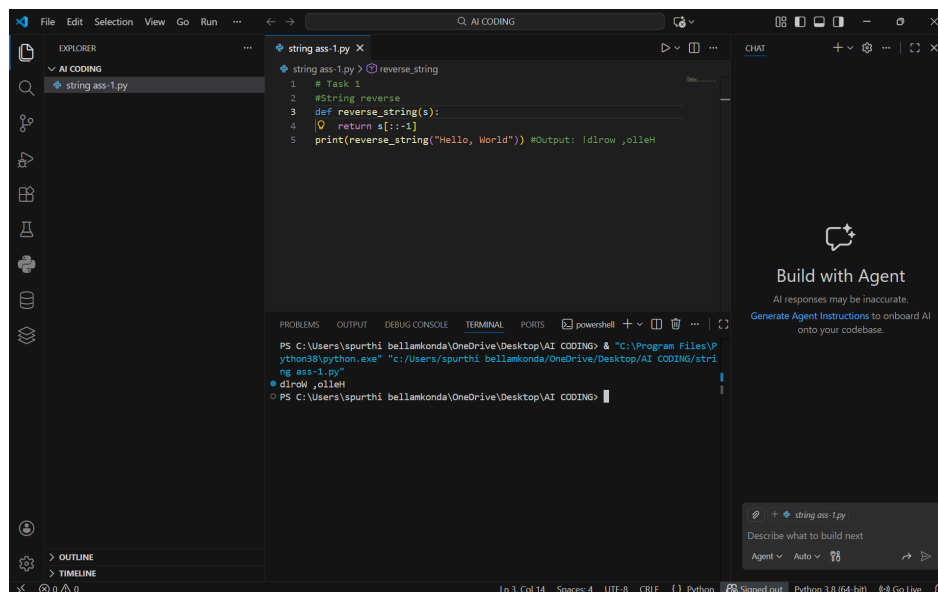
❖ Task Description

Use GitHub Copilot to generate a Python program that:

- Reverses a given string
- Accepts user input
- Implements the logic directly in the main code
- Does not use any user-defined functions

❖ Expected Output

- Correct reversed string
- Screenshots showing Copilot-generated code suggestions
- Sample inputs and outputs



```
# Task 1
#String reverse
def reverse_string(s):
    return s[::-1]
print(reverse_string("Hello, World")) #Output: !dlrow ,olleH
```

This code defines a function named `reverse_string`.

The function takes a string as input.

It uses slicing `[::-1]` to reverse the string.

The reversed string is returned by the function.

Finally, the program prints the reversed output: `!dlrow ,olleH`.

▼ Task 2: Efficiency & Logic Optimization (Readability Improvement)

❖ Scenario

The code will be reviewed by other developers.

❖ Task Description

Examine the Copilot-generated code from Task 1 and improve it by:

- Removing unnecessary variables
- Simplifying loop or indexing logic
- Improving readability
- Use Copilot prompts like:
 - "Simplify this string reversal code"
 - "Improve readability and efficiency"

Hint:

Prompt Copilot with phrases like

"optimize this code", "simplify logic", or "make it more readable"

❖ Expected Output

- Original and optimized code versions
- Explanation of how the improvements reduce time complexity

```
#Task 2
6 def reverse_string(s):
7     """Reverse a string using slicing."""
8     return s[::-1]
9
10 print(reverse_string("Hello, World!")) # Output: !dlroW ,olleH
11
12 # Task 2
13
```

```
6 # Task 2
7 def reverse_string(s):
8     """Reverse a string using slicing."""
9     return s[::-1]
10
11 print(reverse_string("Hello, World!")) # Output: !dlroW ,olleH
12
13 # Task 3
14
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
!dlroW ,olleH
S C:\Users\HP\Desktop\AI> & "C:\Program Files\Python313\python.exe" "c:/Users/HP/Desktop/AI/string ass-1.py"
!dlroW ,olleH
!dlroW ,olleH
S C:\Users\HP\Desktop\AI> & "C:\Program Files\Python313\python.exe" "c:/Users/HP/Desktop/AI/string ass-1.py"
!dlroW ,olleH
!dlroW ,olleH
S C:\Users\HP\Desktop\AI>
```

```
def reverse_string(s):
    """Reverse a string using slicing."""
    return s[::-1]

print(reverse_string("Hello, World!")) # Output: !dlroW ,olleH
```

The code defines a function named `reverse_string`.

It takes one parameter `s`, which is a string.

The line `s[::-1]` reverses the string using Python slicing.

The function returns the reversed string as output.

The print statement calls the function and displays `!dlroW ,olleH`.

```
# Task-4: Comparison Analysis
print("\nTask:4(Comparison Analysis Between Approaches)")
print("\n" + "="*80)
print("ANALYSIS: WITH vs WITHOUT FUNCTIONS")
print("="*80)

comparison_data = {
    "Criteria": ["Code Clarity", "Reusability", "Debugging Ease", "Large-Scale Suitability"],
    "Without Functions": [
        "Low - inline logic is scattered",
        "Poor - must rewrite code each time",
        "Difficult - hard to isolate errors",
        "Not viable - code duplication"
    ],
    "With Functions": [
        "High - logic is encapsulated",
        "Excellent - call function multiple times",
        "Easy - test individual functions",
        "Ideal - modular and maintainable"
    ]
}

# Print comparison table
print(f"\n{'Criteria':<25} {'Without Functions':<35} {'With Functions':<35}")
print("-"*95)
for i, criteria in enumerate(comparison_data["Criteria"]):
    print(f"{'criteria':<25} {'comparison_data['Without Functions'][i]:<35} {'comparison_data['With Functions'][i]:<35}")

print("\n" + "="*80)
print("RECOMMENDATION: Use functions for better maintainability and scalability")
print("="*80)
```

- **Comparison Criteria:**

- **Code Clarity:** Functions make code clearer, while without functions it's scattered.
- **Reusability:** Functions can be reused, but without them you need to rewrite the code.
- **Debugging Ease:** Functions are easier to debug, while without them it's tough.
- **Large-Scale Suitability:** Functions are ideal for large-scale projects, whereas without them, it leads to code duplication.

- **Conclusion:** Use functions for better clarity, reusability, easier debugging, and scalability.

▼ **Task 5: AI-Generated Iterative vs Recursive Fibonacci Approaches (Different Algorithmic Approaches to String Reversal)**

❖ Scenario

Your mentor wants to evaluate how AI handles alternative logic paths.

❖ Task Description

Prompt GitHub Copilot to generate:

- A loop-based string reversal approach
- A built-in / slicing-based string reversal approach

❖ Expected Output

- Two correct implementations
- Comparison discussing:
 - Execution flow
 - Time complexity
 - Performance for large inputs
 - When each approach is appropriate



```

69 # Task-5: Different Algorithmic Approaches to String Reversal
70
71 print("\nTask:5(Different Algorithmic Approaches)")
72 print("\n" + "="*80)
73 print("ALGORITHMIC APPROACHES TO STRING REVERSAL")
74 print("="*80)
75
76 # Approach 1: Loop-Based String Reversal
77 print("\n--- APPROACH 1: LOOP-BASED REVERSAL ---")
78 def reverse_loop(s):
79     """
80     Loop-based approach using iteration
81     Time Complexity: O(n)
82     Space Complexity: O(n)
83     """
84     reversed_str = ""
85     for i in range(len(s) - 1, -1, -1):
86         reversed_str += s[i]
87     return reversed_str
88
89 test_string = "Hello, World!"
90 result1 = reverse_loop(test_string)
91 print(f"Input: {test_string}")
92 print(f"Output: {result1}")
93 print("Execution Flow: Iterate from last index to first, concatenate characters")
94 print("Time Complexity: O(n)")
95 print("Space complexity: O(n)")
96
97 # Approach 2: Built-in Slicing-Based Reversal
98 print("\n--- APPROACH 2: SLICING-BASED REVERSAL ---")
99 def reverse_slicing(s):

```

```

Task:5(Different Algorithmic Approaches)

=====
ALGORITHMIC APPROACHES TO STRING REVERSAL
=====

--- APPROACH 1: LOOP-BASED REVERSAL ---
Input: Hello, World!
Output: !dlroW ,olleH
Execution Flow: Iterate from last index to first, concatenate characters
Time Complexity: O(n)
Space Complexity: O(n)

--- APPROACH 2: SLICING-BASED REVERSAL ---
Input: Hello, World!
Output: !dlroW ,olleH
Execution Flow: Use Python's slice notation with negative step
Time Complexity: O(n)
Space Complexity: O(n)

=====
DETAILED COMPARISON
=====

Criteria          Loop-Based          Slicing-Based
-----
Execution Flow    Manual iteration with index control  Built-in Python slice operation
Time Complexity   O(n) - linear time    O(n) - linear time
Space Complexity  O(n) - new string created  O(n) - new string created
Code Readability  More verbose, explicit logic  Concise and Pythonic
Performance (Small)  Comparable          Comparable
Performance (Large)  Slightly slower due to loop overhead  Faster - optimized C implementation
When to Use       Educational purposes, custom logic needed  Production code, performance-critical applications

=====
RECOMMENDATION: Use slicing (s[::-1]) for production code - it's faster,
cleaner, and leverages Python's optimized built-in operations.
=====

```

Task-5: Different Algorithmic Approaches to String Reversal

```

print("\nTask:5(Different Algorithmic Approaches)")
print("\n" + "="*80)
print("ALGORITHMIC APPROACHES TO STRING REVERSAL")
print("="*80)

```

Approach 1: Loop-Based String Reversal

```

print("\n--- APPROACH 1: LOOP-BASED REVERSAL ---")
def reverse_loop(s):
    """
    Loop-based approach using iteration
    Time Complexity: O(n)
    Space Complexity: O(n)
    """
    reversed_str = ""
    for i in range(len(s) - 1, -1, -1):
        reversed_str += s[i]
    return reversed_str

test_string = "Hello, World!"
result1 = reverse_loop(test_string)
print(f"Input: {test_string}")
print(f"Output: {result1}")
print("Execution Flow: Iterate from last index to first, concatenate characters")
print("Time Complexity: O(n)")
print("Space Complexity: O(n)")

```

Approach 2: Built-in Slicing-Based Reversal

```

print("\n--- APPROACH 2: SLICING-BASED REVERSAL ---")

```

```

def reverse_slicing(s):
    """
    Built-in slicing approach
    Time Complexity: O(n)
    Space Complexity: O(n)
    """
    return s[::-1]

result2 = reverse_slicing(test_string)
print(f"Input: {test_string}")
print(f"Output: {result2}")
print("Execution Flow: Use Python's slice notation with negative step")
print("Time Complexity: O(n)")
print("Space Complexity: O(n)")

# Comparison Analysis
print("\n" + "="*80)
print("DETAILED COMPARISON")
print("="*80)

comparison = {
    "Criteria": ["Execution Flow", "Time Complexity", "Space Complexity", "Code Readability", "Performance (Small)", "Performance (Large)", "When to Use"],
    "Loop-Based": [
        "Manual iteration with index control",
        "O(n) - linear time",
        "O(n) - new string created",
        "More verbose, explicit logic",
        "Comparable",
        "Slightly slower due to loop overhead",
        "Educational purposes, custom logic needed"
    ],
    "Slicing-Based": [
        "Built-in Python slice operation",
        "O(n) - linear time",
        "O(n) - new string created",
        "Concise and Pythonic",
        "Comparable",
        "Faster - optimized C implementation",
        "Production code, performance-critical applications"
    ]
}

print(f"\n{'Criteria':<25} {'Loop-Based':<35} {'Slicing-Based':<35}")
print("-"*95)
for i, criteria in enumerate(comparison["Criteria"]):
    print(f"{criteria:<25} {comparison['Loop-Based'][i]:<35} {comparison['Slicing-Based'][i]:<35}")

print("\n" + "="*80)
print("RECOMMENDATION: Use slicing (s[::-1]) for production code - it's faster,")

```

```
print("cleaner, and leverages Python's optimized built-in operations.")
print("="*80)
```

1. Approach 1 (Loop-Based Reversal):

- **Execution:** Iterates from the last character to the first, building the reversed string.
- **Complexity:** Both time and space complexity are $O(n)$ (linear).
- **When to Use:** Best for educational purposes or custom logic.

2. Approach 2 (Slicing-Based Reversal):

- **Execution:** Uses Python's slicing feature (`[::-1]`) to reverse the string.
- **Complexity:** Time and space complexity are $O(n)$ (linear).
- **When to Use:** Ideal for production code, performance-critical applications.

3. Comparison:

- **Execution Flow:** Loop-based is manual, while slicing is more concise.
- **Time & Space Complexity:** Both approaches have the same $O(n)$ complexities.
- **Performance:** Slicing is generally faster due to Python's internal optimizations.
- **When to Use:** Loop-based for custom logic, slicing for cleaner, optimized production code.

Recommendation: Use slicing (`[::-1]`) for production code—it's faster and cleaner.